



EVALUACIÓN FINAL TRANSVERSAL — DSY1103

Sistema de Reserva de Canchas Deportivas

Arquitectura de microservicios con Spring Boot y Spring Cloud

Equipo (Grupo 8)

Pablo Acuña · Benjamin Cea · Abraham Puente

Contenido

Qué se hizo y cómo está construido el sistema

01 **Objetivo y descripción general**

03 **Los 12 servicios del sistema**

05 **Patrón CSR y comunicación entre servicios**

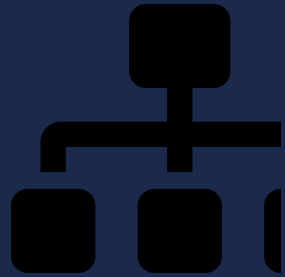
07 **Documentación, despliegue y pruebas**

02 **Arquitectura de microservicios**

04 **Stack tecnológico**

06 **Seguridad, errores y persistencia**

08 **Mi aporte personal: Recinto, Precio y Pago**



Gestionar el proceso completo de reserva

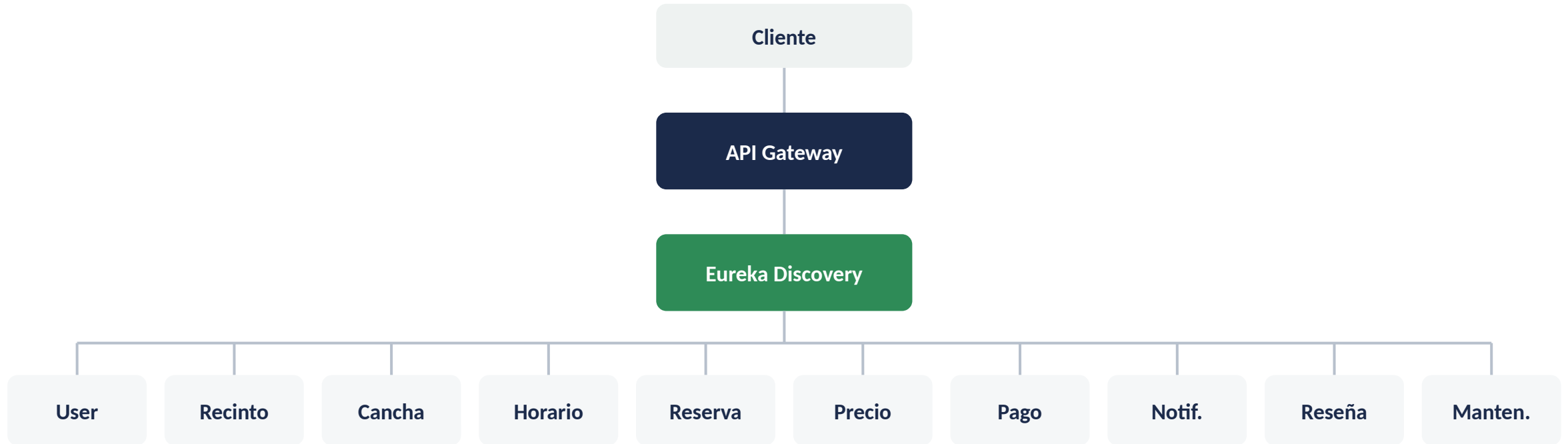
Objetivo del proyecto

Desarrollar una solución distribuida que permita gestionar el ciclo completo de una reserva de cancha deportiva: desde la creación del usuario y el recinto hasta el pago, la reseña, el mantenimiento y la notificación.

- Arquitectura de microservicios
- Patrón Controller-Service-Repository
- Persistencia con JPA e Hibernate
- Migraciones con Flyway
- Seguridad con Spring Security
- Descubrimiento con Eureka
- API Gateway centralizado
- Documentación Swagger/OpenAPI
- Contenedores Docker
- Despliegue remoto

Arquitectura general

Punto único de entrada + descubrimiento dinámico de servicios



El API Gateway es el punto único de entrada y enruta cada petición al microservicio correspondiente. **Eureka** permite que los servicios se localicen por nombre lógico (ej. RECINTO-SERVICE) en vez de depender de IPs o localhost fijos.

Los 12 servicios del sistema

10 microservicios de negocio + Discovery Service + API Gateway



User Service

Usuarios, perfiles y roles



Recinto Service

Recintos deportivos



Cancha Service

Canchas por recinto



Horario Service

Horarios disponibles



Reserva Service

Creación y gestión de reservas



Precio Service

Precios por cancha



Pago Service

Registro y validación de pagos



Notificación Service

Notificaciones a usuarios



Reseña Service

Comentarios y calificaciones



Mantenimiento Service

Mantenimientos de canchas

Stack tecnológico



Backend

- Java 17
- Spring Boot
- Spring Data JPA + Hibernate
- Spring Validation
- Spring Security
- Spring HATEOAS
- RestClient



Arquitectura distribuida

- Netflix Eureka
- Spring Cloud Gateway
- Spring Cloud LoadBalancer



Persistencia

- MySQL
- H2 (dev/test local)
- Flyway (migraciones)



Documentación

- Swagger / OpenAPI
- Centralizada en el Gateway



Despliegue

- Docker + Docker Compose
- Render (servicios)
- Railway (bases de datos)



Gestión de proyecto

- Maven
- Git + GitHub

Patrón CSR y organización del código

Cada microservicio separa responsabilidades en tres capas



- DTOs de entrada y salida separados de las entidades JPA.
- GlobalExceptionHandler (@RestControllerAdvice) para manejo centralizado de errores.
- Bean Validation (JSR 380) en los DTOs de entrada.
- Logs estructurados con SLF4J en creación, actualización y errores.

Comunicación entre microservicios

RestClient + LoadBalancer

Cuando un servicio necesita validar información de otro, hace una llamada REST sincrónica usando el nombre lógico registrado en Eureka en vez de una URL fija.

- Cancha consulta la existencia de un recinto.
- Horario consulta la existencia de una cancha.
- Reserva valida usuario, cancha y horario.
- Precio valida que la cancha exista.
- Pago valida que la reserva exista.
- Reseña valida usuario y cancha.
- Mantenimiento valida una cancha.

Ejemplo — nombres Eureka

`http://USER-SERVICE`

`http://RECINTO-SERVICE`

`http://CANCHASERVICE`

`http://HORARIO-SERVICE`

`http://RESERVA-SERVICE`

Ventaja: se evitan URLs fijas (localhost / IP) entre microservicios; Eureka resuelve la dirección real de cada instancia en tiempo de ejecución.

Seguridad y roles

Spring Security con autenticación Basic Auth



ADMIN

Administración general del sistema.



DUEÑO

Administra recintos, canchas y mantenimientos.



CLIENTE

Crea reservas, pagos y reseñas.

Usuarios de prueba

Usuario	Contraseña	Rol
admin	admin123	ADMIN
duenio	duenio123	DUEÑO
cliente	cliente123	CLIENTE

Manejo centralizado de errores

GlobalExceptionHandler (@RestControllerAdvice) en cada microservicio

400

Bad Request

Regla de negocio o validación fallida

401

Unauthorized

Sin autenticación válida

403

Forbidden

Rol sin permiso para la acción

404

Not Found

Recurso no encontrado

500

Internal Error

Error no controlado

Ejemplo de respuesta ante una regla de negocio violada

```
{
  "error": "Business Rule Violation",
  "message": "Solo un usuario CLIENTE o ADMIN
              puede crear reservas",
  "timestamp": "2026-07-12T01:41:25",
  "status": 400
}
```

- ResourceNotFoundException → 404
- BusinessException → 400
- MethodArgumentNotValidException → 400 con lista de errores de validación
- Formato de respuesta consistente en todos los microservicios

Persistencia y bases de datos



Una base de datos por servicio

- MySQL en local, mediante Docker.
- MySQL en la nube, alojado en Railway para el entorno remoto.
- H2 para desarrollo y pruebas locales en algunos servicios.
- Cada base de datos es independiente: separa la persistencia por dominio de negocio.



Migraciones con Flyway

Cada microservicio guarda sus scripts de migración en:

```
src/main/resources/db/migration
```

Flyway valida y aplica las migraciones automáticamente al iniciar cada servicio (ej. V1__create_tables.sql).

¿Por qué Render + Railway por separado? Render ejecuta los servicios web; Railway aloja las bases de datos. Así la información se conserva aunque un servicio se reinicie o vuelva a desplegarse.

Documentación y despliegue



Swagger / OpenAPI centralizado

Cada microservicio documenta sus endpoints con springdoc-openapi. El API Gateway agrega todas las documentaciones en un solo Swagger UI.

```
/swagger-ui/index.html
```

Desde ahí se pueden consultar y probar en vivo los endpoints de los 10 microservicios de negocio, tanto en local como en el entorno remoto.



Dos formas de ejecutar el sistema

Local — Docker Compose

```
docker compose up -d --build
```

Levanta los 12 servicios + MySQL local en un solo comando.

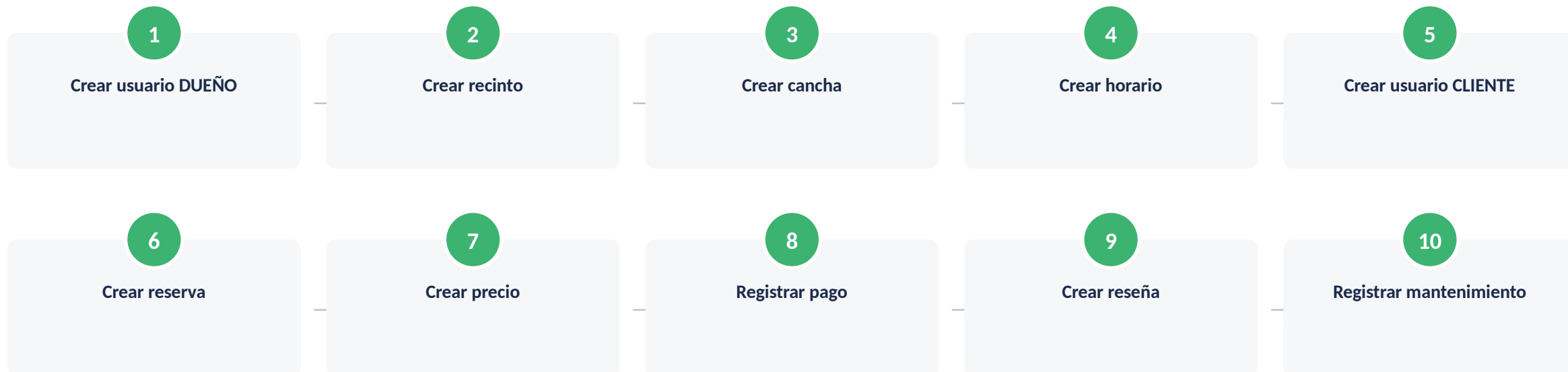
Remoto — Render + Railway

Cada servicio se despliega como Web Service independiente en Render; las bases de datos MySQL viven en Railway. La configuración se maneja por variables de entorno.

wake-render.sh reactiva Eureka, los microservicios y el Gateway cuando el plan gratuito los suspende por inactividad.

Flujo de prueba recomendado

Probado de extremo a extremo mediante el Swagger centralizado



Mi aporte personal

Desarrollé Recinto Service, Precio Service y Pago Service



Recinto Service

Puerto 8082

Registro de recintos deportivos

Valida al dueño contra User Service · no permite recintos duplicados (nombre + dirección + comuna)



Precio Service

Puerto 8086

Precio asociado a cada cancha

Valida que la cancha exista · una cancha no puede tener más de un precio ACTIVO



Pago Service

Puerto 8087

Registro de pagos de una reserva

Valida que la reserva exista · método de pago y estado controlados · una reserva no puede pagarse dos veces

Estado final del proyecto

Ecosistema desplegado y operativo — validado de punta a punta

- ✓ Registro de los 10 servicios de negocio en Eureka
- ✓ Swagger centralizado funcionando
- ✓ Persistencia en bases de datos MySQL independientes
- ✓ Seguridad y roles (ADMIN / DUEÑO / CLIENTE)
- ✓ Ejecución local vía Docker Compose
- ✓ Enrutamiento correcto mediante el API Gateway
- ✓ Comunicación estable entre microservicios
- ✓ Migraciones Flyway aplicadas en cada servicio
- ✓ Reglas de negocio validadas en los 10 servicios
- ✓ Despliegue remoto en Render + Railway



Gracias

Sistema de Reserva de Canchas Deportivas — DSY1103

github.com/abrahampuente/Proyecto-reserva-cancha

Pablo Acuña · Benjamin Cea · Abraham Puente